

Fully Connected Neural Networks

2025. 06. 02 AI Expert

Chaehun Shin

Data Science and Artificial Intelligence Laboratory

Seoul National University



Objectives

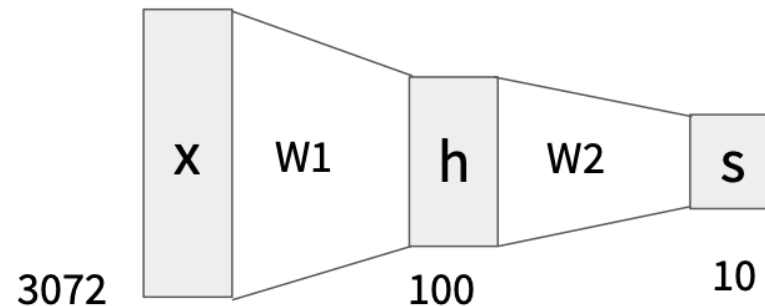
- CIFAR-10 dataset에 대한 Softmax 분류기의 고도화
- 분류기 학습을 위한 손실 함수 구현 및 최적화 이해

Two-layer Neural Networks

- Two-layer Classifier $\Rightarrow$ Forward / Backward propagation

$$f = Wx + b \rightarrow f = W_2 \max(0, W_1x + b_1) + b_2$$

$$x \in R^D, W_1 \in R^{H \times D}, W_2 \in R^{C \times H}, b_1 \in R^H, b_2 \in R^C$$



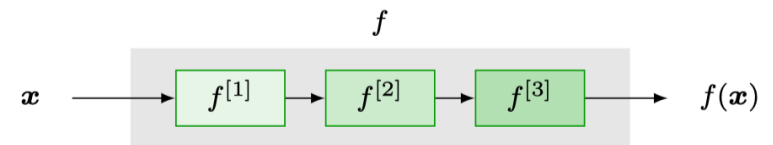
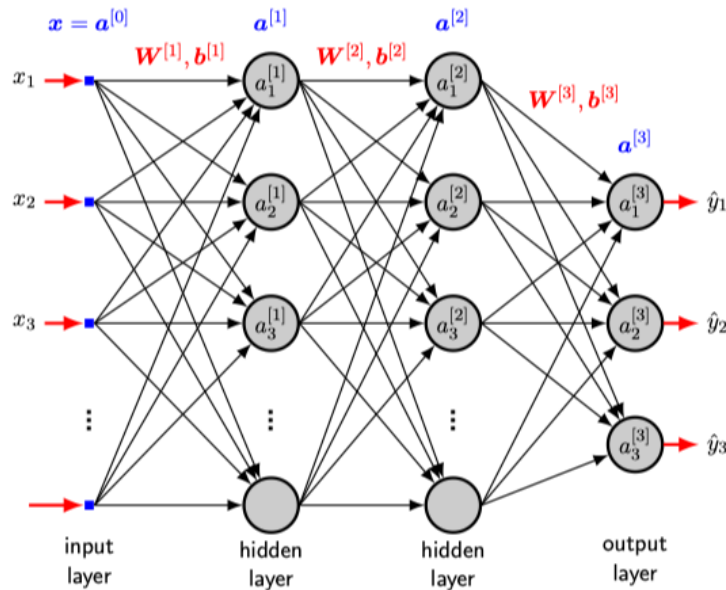
- Score = $f(x) = W_2 \max(0, W_1x + b_1) + b_2$, probability = $\text{softmax}(f(x)) = \frac{\exp(f(x))}{\sum \exp(f(x))[j]}$
- $L = -\log(\text{probability of correct class}) + \text{reg}(w) = -\log\left(\frac{\exp(f(x))[k]}{\sum \exp(f(x))[j]}\right) + \lambda \|w\|_2^2 = -\sum y_i \log(\sigma_i) + \lambda \|w\|_2^2$
- $W_1 = W_1 - \epsilon \nabla_{w_1} L, W_2 = W_2 - \epsilon \nabla_{w_2} L, b_1 = b_1 - \epsilon \nabla_{b_1} L, b_2 = b_2 - \epsilon \nabla_{b_2} L$
- $\nabla_{w_2} L = h^T(\hat{y} - y) + 2\lambda W_2, \nabla_{b_2} L = \hat{y} - y$
 $da = (\hat{y} - y)W_2^T \odot [a > 0] \ (a = W_1x + b), \nabla_{w_1} L = X^T da + 2\lambda W_1, \nabla_{b_1} L = da$

Fully Connected Neural Networks

- How can we make more complex neural networks?
 - K개의 layer를 쌓아서 더욱 복잡한 neural network 구현

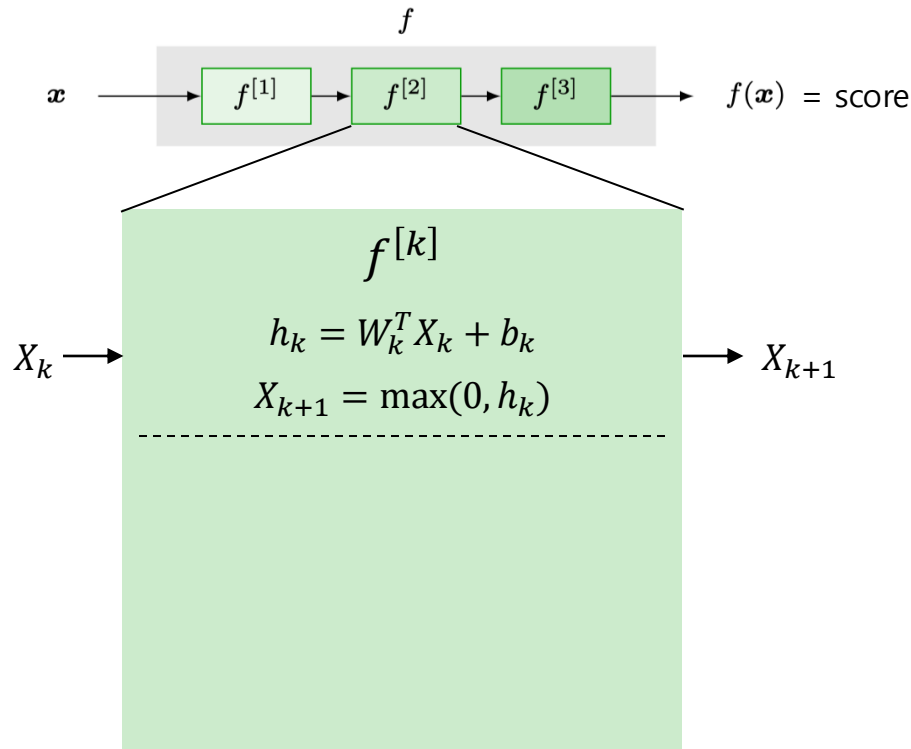
$$f = Wx + b \rightarrow f = W_2 \max(0, W_1 x + b_1) + b_2$$

$$\rightarrow f = W_k \max(0, \dots (W_2 \max(0, W_1 x + b_1) + b_2) \dots) + b_k$$



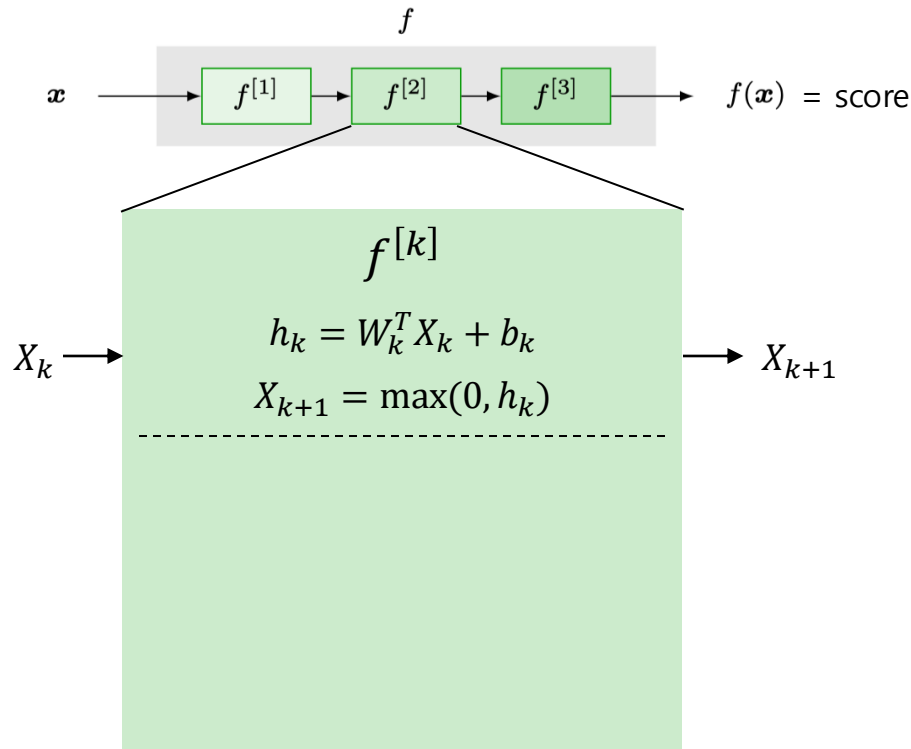
Fully Connected Neural Networks

- 각 layer들의 module화 = affine function + relu function
 - Forward function
 - 이전 layer에서 넘어온 Input에서 해당 layer의 weight W 를 곱하고 bias b 를 더해준 다음 non-linear activation을 통과해 output을 다음 layer로



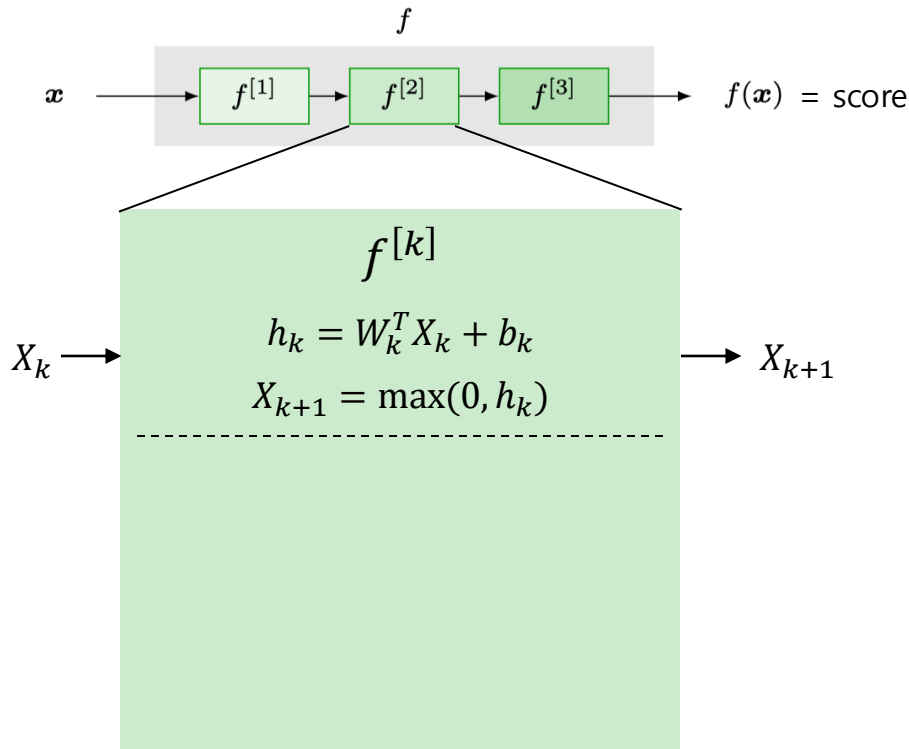
Fully Connected Neural Networks

- 각 layer들의 module화 = affine function + relu function
 - Forward function
 - 이전 layer에서 넘어온 Input에서 해당 layer의 weight W 를 곱하고 bias b 를 더해준 다음 non-linear activation을 통과해 output을 다음 layer로



Fully Connected Neural Networks

- 각 layer들의 module화 = affine function + relu function
 - Forward function
 - 이전 layer에서 넘어온 Input에서 해당 layer의 weight W 를 곱하고 bias b 를 더해준 다음



```
def affine_forward(x, w, b):
```

```
    """
    Computes the forward pass for an affine (fully-connected) layer.
```

```
    The input x has shape (N, d_1, ..., d_k) and contains a minibatch of N
    examples, where each example x[i] has shape (d_1, ..., d_k). We will
    reshape each input into a vector of dimension D = d_1 * ... * d_k, and
    then transform it to an output vector of dimension M.
```

```
    Inputs:
```

```
    - x: A numpy array containing input data, of shape (N, d_1, ..., d_k)
    - w: A numpy array of weights, of shape (D, M)
    - b: A numpy array of biases, of shape (M,)
```

```
    Returns a tuple of:
```

```
    - out: output, of shape (N, M)
    - cache: (x, w, b)
```

```
    """
```

```
    out = None
```

```
    num_train = x.shape[0] #number of training examples
```

```
    #####
    # TODO: Implement the affine forward pass. Store the result in out. You #
    # will need to reshape the input into rows.                          #
    #####
```

```
    #####
    #                               END OF YOUR CODE                      #
    #####
```

```
    cache = (x, w, b)
```

```
    return out, cache
```

```
def relu_forward(x):
```

```
    """
```

```
    Computes the forward pass for a layer of rectified linear units (ReLU).
```

```
    Input:
```

```
    - x: Inputs, of any shape
```

```
    Returns a tuple of:
```

```
    - out: Output, of the same shape as x
```

```
    - cache: x
```

```
    """
```

```
    out = None
```

```
    #####
    # TODO: Implement the ReLU forward pass.                              #
    #####
```

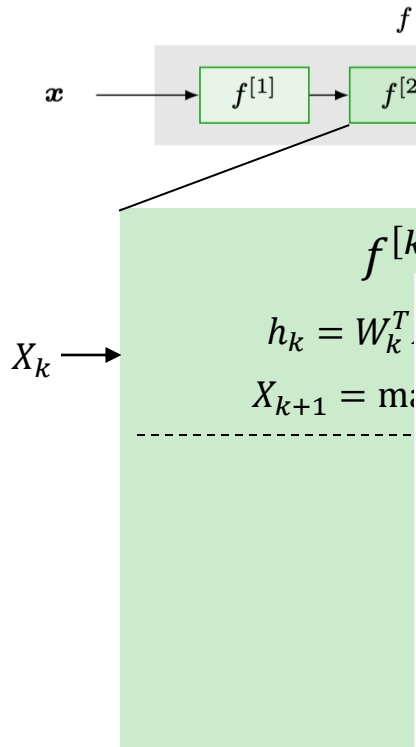
```
    #####
    #                               END OF YOUR CODE                      #
    #####
```

```
    cache = x
```

```
    return out, cache
```

Fully Connected

- 각 layer들의 module
 - Forward function
 - 이전 layer에서 나온 output을 다음 layer로 전달하는 역할
 - 해당 layer의 weights와 biases를 사용하여 output을 계산



```
def affine_forward(x, w, b):
    """
    Computes the forward pass for an affine (fully-connected) layer.

    The input x has shape (N, d_1, ..., d_k) and contains a minibatch of N
    examples, where each example x[i] has shape (d_1, ..., d_k). We will
    reshape each input into a vector of dimension D = d_1 * ... * d_k, and
    then transform it to an output vector of dimension M.

    Inputs:
    - x: A numpy array containing input data, of shape (N, d_1, ..., d_k)
    - w: A numpy array of weights, of shape (D, M)
    - b: A numpy array of biases, of shape (M,)

    Returns a tuple of:
    - out: output, of shape (N, M)
    - cache: (x, w, b)
    """
    out = None
    num_train = x.shape[0] #number of training examples
    #####
    # TODO: Implement the affine forward pass. Store the result in out. You #
    # will need to reshape the input into rows. #####
    out = np.dot(x.reshape(num_train, -1), w) + b
    #####
    #                               END OF YOUR CODE                               #
    #####
    cache = (x, w, b)
    return out, cache

def relu_forward(x):
    """
    Computes the forward pass for a layer of rectified linear units (ReLU).

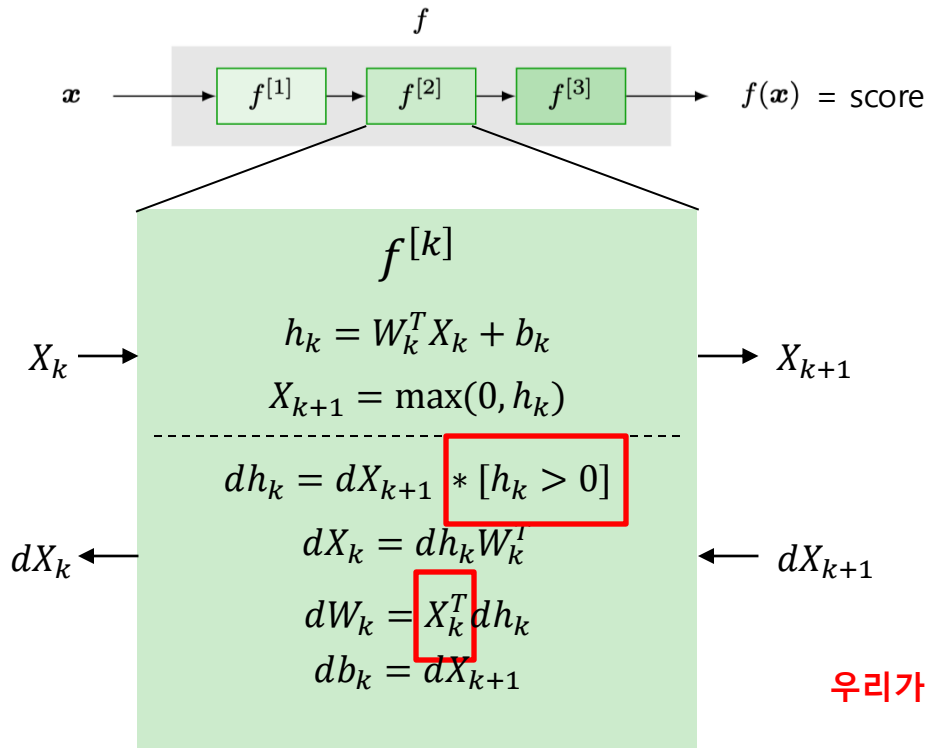
    Input:
    - x: Inputs, of any shape

    Returns a tuple of:
    - out: Output, of the same shape as x
    - cache: x
    """
    out = None
    #####
    # TODO: Implement the ReLU forward pass. #####
    out = np.maximum(0, x) #ReLU functions
    #####
    #                               END OF YOUR CODE                               #
    #####
    cache = x
    return out, cache
```

output을 다음 layer로

Fully Connected Neural Networks

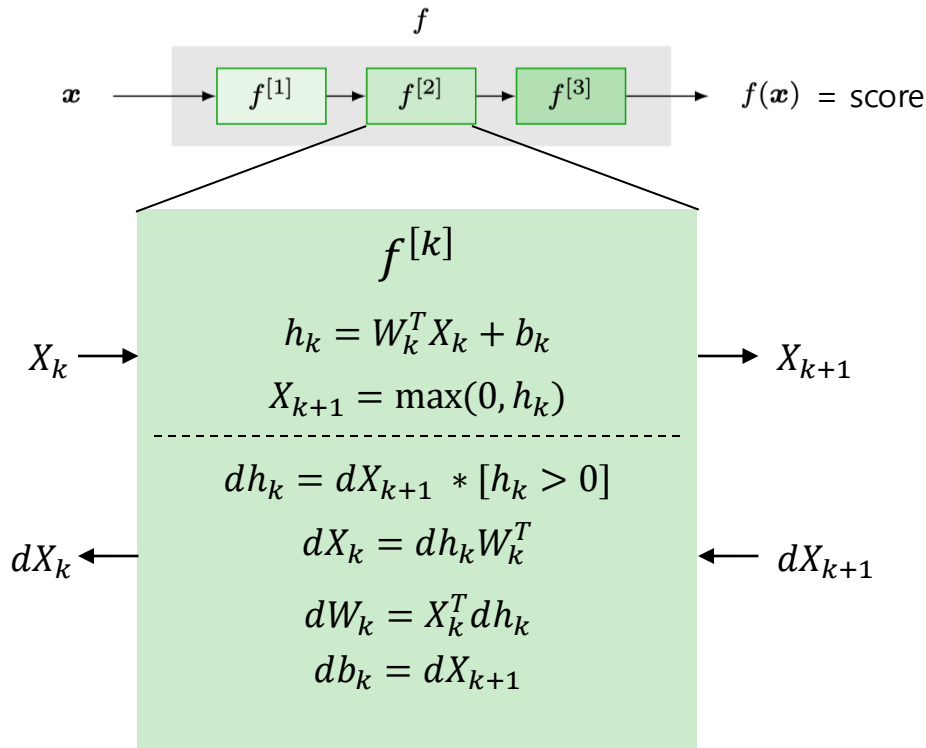
- 각 layer들의 module화 = affine function + relu function
 - Backward function
 - 반대로 output gradient에서 각 연산들에 대한 미분을 통해
해당 layer의 weight W, bias b에 대한 gradient를 구해서 업데이트 & input gradient를 구해서 이전 layer로 넘겨줌



우리가 forward에서 caching을 해줘야 했던 이유

Fully Connected Neural Networks

- 각 layer들의 module화 = affine function + relu function
 - Backward function
 - 반대로 output gradient에서 각 연산들에 대한 미분을 통해 해당 layer의 weight W, bias b에 대한 gradient를 구해서



```
def affine_backward(dout, cache):
    """
    Computes the backward pass for an affine layer.

    Inputs:
    - dout: Upstream derivative, of shape (N, M)
    - cache: Tuple of:
      - x: Input data, of shape (N, d_1, ..., d_k)
      - w: Weights, of shape (D, M)

    Returns a tuple of:
    - dx: Gradient with respect to x, of shape (N, d_1, ..., d_k)
    - dw: Gradient with respect to w, of shape (D, M)
    - db: Gradient with respect to b, of shape (M,)
    """
    x, w, b = cache
    num_train = x.shape[0]
    dx, dw, db = None, None, None
    #####
    # TODO: Implement the affine backward pass.
    #####
    #####
    #                               END OF YOUR CODE
    #####
    return dx, dw, db

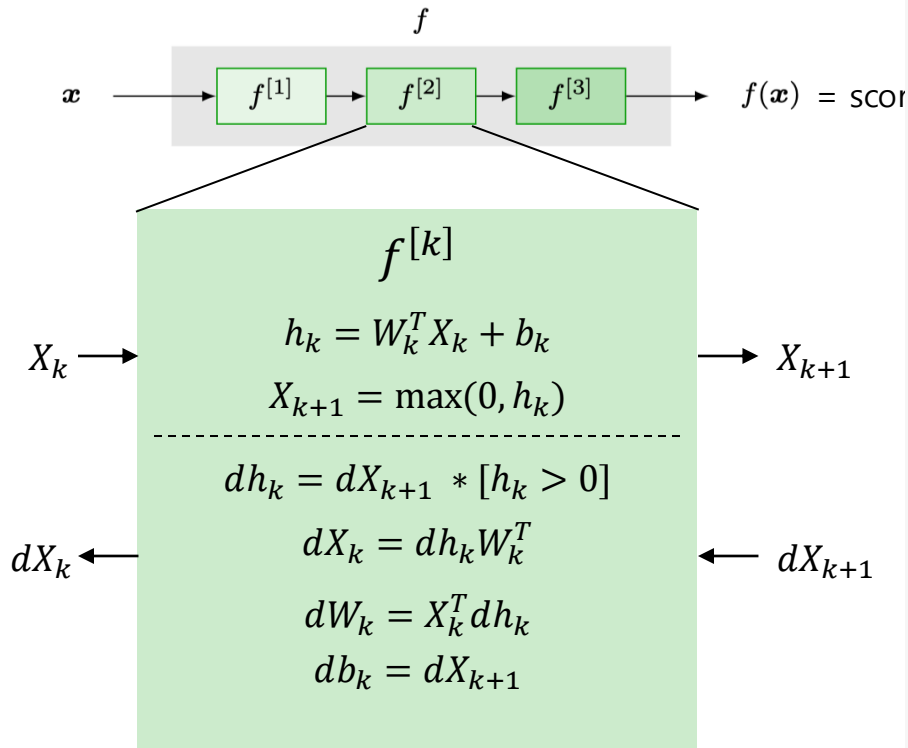
def relu_backward(dout, cache):
    """
    Computes the backward pass for a layer of rectified linear units (ReLUs).

    Input:
    - dout: Upstream derivatives, of any shape
    - cache: Input x, of same shape as dout

    Returns:
    - dx: Gradient with respect to x
    """
    dx, x = None, cache
    #####
    # TODO: Implement the ReLU backward pass.
    #####
    #####
    #                               END OF YOUR CODE
    #####
    return dx
```

Fully Connected Neural Network

- 각 layer들의 module화 = affine function + relu function
 - Backward function
 - 반대로 output gradient에서 각 연산들에 대한 미분
해당 layer의 weight W, bias b에 대한 gradient를 구



```
def affine_backward(dout, cache):
    """
    Computes the backward pass for an affine layer.

    Inputs:
    - dout: Upstream derivative, of shape (N, M)
    - cache: Tuple of:
      - x: Input data, of shape (N, d_1, ... d_k)
      - w: Weights, of shape (D, M)

    Returns a tuple of:
    - dx: Gradient with respect to x, of shape (N, d1, ..., d_k)
    - dw: Gradient with respect to w, of shape (D, M)
    - db: Gradient with respect to b, of shape (M,)
    """
    x, w, b = cache
    num_train = x.shape[0]
    dx, dw, db = None, None, None

    #####
    # TODO: Implement the affine backward pass.
    #####
    db = np.sum(dout, axis = 0)
    dx = np.dot(dout, w.T).reshape(*x.shape) #to match dimensions of x
    dw = np.dot(x.reshape(num_train, -1).T, dout)

    #####
    #                               END OF YOUR CODE
    #####
    return dx, dw, db

def relu_backward(dout, cache):
    """
    Computes the backward pass for a layer of rectified linear units (ReLU).

    Input:
    - dout: Upstream derivatives, of any shape
    - cache: Input x, of same shape as dout

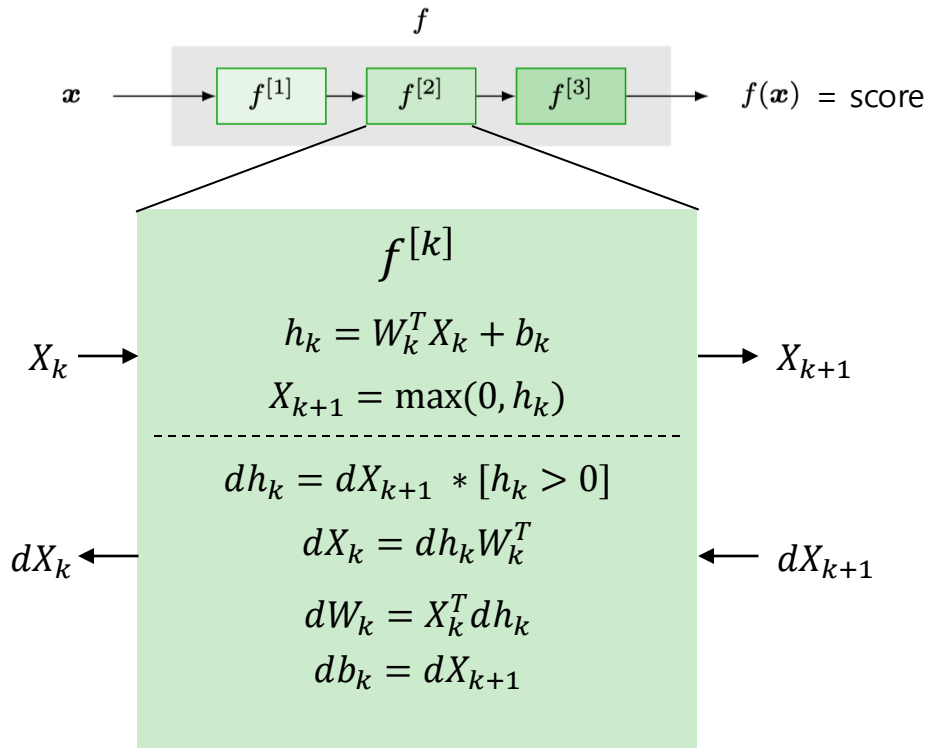
    Returns:
    - dx: Gradient with respect to x
    """
    dx, x = None, cache

    #####
    # TODO: Implement the ReLU backward pass.
    #####
    dx = np.zeros(x.shape)
    dx[x>0] = 1
    dx = dx * dout

    #####
    #                               END OF YOUR CODE
    #####
    return dx
```

Fully Connected Neural Networks

- 전체 network의 구현
 - Layer module을 차원만 맞춘 형태에서 반복적으로 stacking
 - 마지막 layer에서는 class 갯수만큼 output 되도록 해서 score를 얻어냄
 - 이로부터 softmax를 이용해 확률값 계산



Fully Con

- 전체 network
 - Layer mo
 - 마지막 la
 - 이로부터

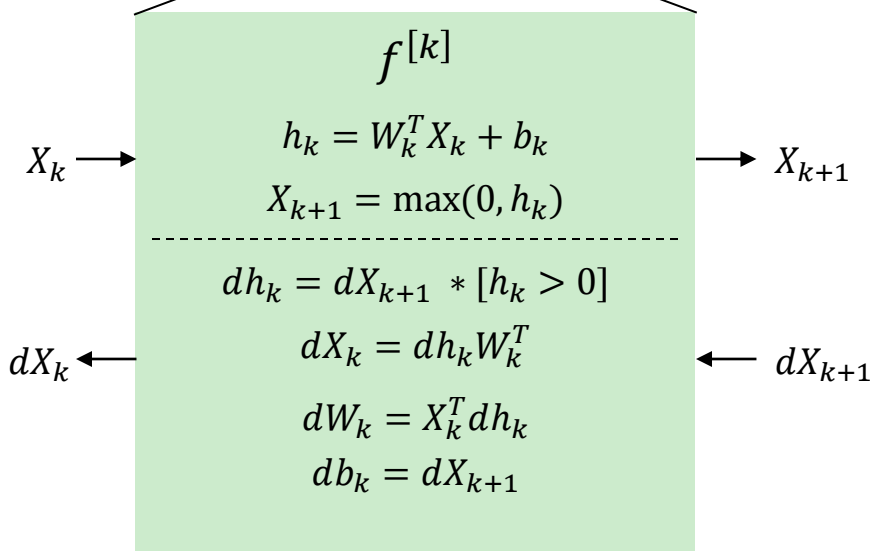


```
def __init__(self, hidden_dims, input_dim=3*32*32, num_classes=10,
              dropout=0, use_batchnorm=False, reg=0.0,
              weight_scale=1e-2, dtype=np.float32, seed=None):
    """
    Initialize a new FullyConnectedNet.

    Inputs:
    - hidden_dims: A list of integers giving the size of each hidden layer.
    - input_dim: An integer giving the size of the input.
    - num_classes: An integer giving the number of classes to classify.
    - dropout: Scalar between 0 and 1 giving dropout strength. If dropout=0 then
      the network should not use dropout at all.
    - use_batchnorm: Whether or not the network should use batch normalization.
    - reg: Scalar giving L2 regularization strength.
    - weight_scale: Scalar giving the standard deviation for random
      initialization of the weights.
    - dtype: A numpy datatype object; all computations will be performed using
      this datatype. float32 is faster but less accurate, so you should use
      float64 for numeric gradient checking.
    - seed: If not None, then pass this random seed to the dropout layers. This
      will make the dropout layers deterministic so we can gradient check the
      model.
    """
    self.use_batchnorm = use_batchnorm
    self.use_dropout = dropout > 0
    self.reg = reg
    self.num_layers = 1 + len(hidden_dims)
    self.dtype = dtype
    self.params = {}

    dims = [input_dim] + hidden_dims + [num_classes] #all the dimensions of network

    for i in range(self.num_layers): #for each layer
        self.params["W" + str(i+1)] = np.random.normal(scale = weight_scale, size = (dims[i], dims[i+1]))
        self.params["b" + str(i+1)] = np.zeros(dims[i+1])
```



Fully Connected Neural Networks

- 전체 network의 구현
 - Layer module을 차원만 맞춘 형태에서 반복적으로 stacking

```
def loss(self, X, y=None):
    """
    Compute loss and gradient for the fully-connected net.

    Input / output: Same as TwoLayerNet above.
    """
    X = X.astype(self.dtype)
    mode = 'test' if y is None else 'train'

    # Set train/test mode for batchnorm params and dropout param since they
    # behave differently during training and testing.
    if self.use_dropout:
        self.dropout_param['mode'] = mode
    if self.use_batchnorm:
        for bn_param in self.bn_params:
            bn_param['mode'] = mode

    scores = None
    layer_input = X
    caches = [] #for backprop
    rl_caches = []
    if (self.use_dropout):
        dp_caches = []
    if (self.use_batchnorm):
        bn_caches = []
    for i in range(self.num_layers - 1): #last layer is just affine_forward
        layer_input, cache = affine_forward(layer_input, self.params["W" + str(i+1)], self.params["b" + str(i+1)])
        caches.append(cache)
        if (self.use_batchnorm):
            layer_input, cache = batchnorm_forward(layer_input, self.params["gamma" + str(i+1)], self.params["beta" + str(i+1)])
            bn_caches.append(cache)
        layer_input, cache = relu_forward(layer_input)
        rl_caches.append(cache)
        if (self.use_dropout):
            layer_input, d_cache = dropout_forward(layer_input, self.dropout_param)
            dp_caches.append(d_cache)
    scores, cache = affine_forward(layer_input, self.params["W" + str(self.num_layers)], self.params["b" + str(self.num_layers)])
    caches.append(cache)

    # If test mode return early
    if mode == 'test':
        return scores
```

해서 score를 얻어냄

```
loss, grads = 0.0, {}
#for the last layer
loss, dloss = softmax_loss(scores, y)
dloss, grads["W" + str(self.num_layers)], grads["b" + str(self.num_layers)] = affine_backward(dloss, caches[-1])
loss += 0.5 * self.reg * np.sum(self.params["W" + str(self.num_layers)] * self.params["W" + str(self.num_layers)])
grads["W" + str(self.num_layers)] += self.reg * self.params["W" + str(self.num_layers)]

for i in range(self.num_layers-1, 0, -1): #from backward in the list and with which layer
    #dropout backward
    if (self.use_dropout):
        dloss = dropout_backward(dloss, dp_caches[i-1])
    #relu backward
    dloss = relu_backward(dloss, rl_caches[i-1])
    #batchnorm backward
    if (self.use_batchnorm):
        dloss, grads["gamma" + str(i)], grads["beta" + str(i)] = batchnorm_backward(dloss, bn_caches[i-1])
    #finally affine backward
    dloss, grads["W" + str(i)], grads["b" + str(i)] = affine_backward(dloss, caches[i-1])
    loss += 0.5 * self.reg * np.sum(self.params["W" + str(i)] * self.params["W" + str(i)])
    grads["W" + str(i)] += self.reg * self.params["W" + str(i)]

return loss, grads
```

$$dX_k \leftarrow \begin{matrix} dX_k = dh_k W_k^T \\ dW_k = X_k^T dh_k \\ db_k = dX_{k+1} \end{matrix} \leftarrow dX_{k+1}$$

Fully Connected Neural Networks

- 학습 코드
 - 구현한 Modulated forward / backward를 바탕으로 쌓은 network에서
 - 매 iteration마다 batch size만큼의 data를 선택한 뒤,
 - modulated forward / backward를 수행해서 weight를 update

```

def train(self):
    """
    Run optimization to train the model.
    """
    num_train = self.X_train.shape[0]
    iterations_per_epoch = max(num_train // self.batch_size, 1)
    num_iterations = self.num_epochs * iterations_per_epoch

    for t in range(num_iterations):
        self._step()

        # Maybe print training loss
        if self.verbose and t % self.print_every == 0:
            print('(Iteration %d / %d) loss: %f' % (
                t + 1, num_iterations, self.loss_history[-1]))

        # At the end of every epoch, increment the epoch counter and decay
        # the learning rate.
        epoch_end = (t + 1) % iterations_per_epoch == 0
        if epoch_end:
            self.epoch += 1
            for k in self.optim_configs:
                self.optim_configs[k]['learning_rate'] *= self.lr_decay

        # Check train and val accuracy on the first iteration, the last
        # iteration, and at the end of each epoch.
        first_it = (t == 0)
        last_it = (t == num_iterations - 1)
        if first_it or last_it or epoch_end:
            train_acc = self.check_accuracy(self.X_train, self.y_train,
                num_samples=self.num_train_samples)
            self.train_acc_history.append(train_acc)
            if self.X_val is not None:
                val_acc = self.check_accuracy(self.X_val, self.y_val,
                    num_samples=self.num_val_samples)
                self.val_acc_history.append(val_acc)

```


Get your best Fully Connected Neural Networks

- 다양한 size의 network 구현 및 hyperparameter setting 후에 학습
 - 몇개의 layer와 어떤 Hidden dimension을 사용할지
 - Learning rate와 regularization weight, optimizer는 어떤 것을 사용할 것인지

Get your best Fully Connected Neural Networks

- ```

TODO: Train the best FullyConnectedNet that you can on CIFAR-10. You might #
batch normalization and dropout useful. Store your best model in the #
best_model variable. #

hidden_dims = [] # TODO : set your hidden dimension list
optim_type = # TODO : set your optimizer
lr = # TODO : set your learning rate
batch_size = # TODO : set your batch size
reg_weight = # TODO : set your regularization weight
num_epochs = # TODO : set your training epoch

model = FullyConnectedNet(hidden_dims = hidden_dims, weight_scale=5e-2, reg = reg_weight)
solver = Solver(model, data, update_rule=optim_type, optim_config={'learning_rate': lr,}, lr_decay=0.95,
 num_epochs = num_epochs, batch_size = batch_size, verbose = True)
solver.train()

print ("validation accuracy is: " + str(solver.best_val_acc))

if (best_val_acc < solver.best_val_acc):
 best_model = model
 best_val_acc = solver.best_val_acc

print ("Better Validation accuracy is: " + str(best_val_acc))

END OF YOUR CODE #
#####
```

# Pytorch Implementation

- 매번 이러한 layer module의 forward / backward function을 구현하고 이를 쌓아서 연산?
- 굉장히 복잡하고 번거로운 작업
  - 특히 gradient descent를 계산하기 위해 미분하는 법을 직접 구해서 수식에 따라 구현하는 것은 굉장히 귀찮음
- 모든 연산들을 모듈화 해서 다 구현 해놓은 GPU 지원이 가능한 library = **Pytorch**

# Pytorch Implementation

- Pytorch implementation vs Numpy implementation (ours)

```
class TwoLayerFC(nn.Module):
 def __init__(self, input_size, hidden_size, num_classes):
 super().__init__()
 # assign layer objects to class attributes
 self.fc1 = nn.Linear(input_size, hidden_size)
 # nn.init package contains convenient initialization methods
 # http://pytorch.org/docs/master/nn.html#torch-nn-init
 nn.init.kaiming_normal_(self.fc1.weight)
 self.fc2 = nn.Linear(hidden_size, num_classes)
 nn.init.kaiming_normal_(self.fc2.weight)

 def forward(self, x):
 # forward always defines connectivity
 x = flatten(x)
 scores = self.fc2(F.relu(self.fc1(x)))
 return scores

def test_TwoLayerFC():
 input_size = 50
 x = torch.zeros((64, input_size), dtype=dtype) # minibatch size 64, feature dimension 50
 model = TwoLayerFC(input_size, 42, 10)
 scores = model(x)
 print(scores.size()) # you should see [64, 10]
test_TwoLayerFC()
```

```
def loss(self, X, y=None):
 """
 Compute the loss and gradients for a two layer fully connected neural
 network.

 Inputs:
 - X: Input data of shape (N, D). Each X[i] is a training sample.
 - y: Vector of training labels. y[i] is the label for X[i], and each y[i] is
 an integer in the range 0 <= y[i] < C. This parameter is optional; if it
 is not passed then we only return scores, and if it is passed then we
 instead return the loss and gradients.

 Returns:
 If y is None, return a matrix scores of shape (N, C) where scores[i, c] is
 the score for class c on input X[i].

 If y is not None, instead return a tuple of:
 - loss: Loss (data loss and regularization loss) for this batch of training
 samples.
 - grads: Dictionary mapping parameter names to gradients of those parameters
 with respect to the loss function; has the same keys as self.params.
 """
 # Unpack variables from the params dictionary
 W1, b1 = self.params['W1'], self.params['b1']
 W2, b2 = self.params['W2'], self.params['b2']
 N, D = X.shape

 # Compute the forward pass
 scores = None
 #####
 # TODO: Perform the forward pass, computing the class scores for the input. #
 # Store the result in the scores variable, which should be an array of #
 # shape (N, C). #
 #####
 Z1 = X.dot(W1) + b1
 O1 = np.maximum(0, Z1)
 scores = O1.dot(W2) + b2
 #####
 # END OF YOUR CODE #
 #####
 # TODO: Finish the forward pass, and compute the loss. This should include #
 # both the data loss and L2 regularization for W1 and W2. Store the result #
 # in the variable loss, which should be a scalar. Use the Softmax #
 # classifier loss. #
 #####
 exp_scores = np.exp(scores)
 probabilities = exp_scores / np.sum(exp_scores, axis = 1, keepdims = True)

 loss = -np.sum(np.log(probabilities[range(N), y])) #data loss
 loss /= N
 loss += self.reg * np.sum(W1*W1)
 loss += self.reg * np.sum(W2*W2)
 #####
 # END OF YOUR CODE #
 #####

 # Backward pass: compute gradients
 grads = {}
 #####
 # TODO: Compute the backward pass, computing the derivatives of the weights #
 # and biases. Store the results in the grads dictionary. For example, #
 # grads['W1'] should store the gradient on W1, and be a matrix of same size #
 #####
 grad_Z2 = probabilities
 grad_Z2[range(N), y] -= 1.0
 grad_Z2 /= N

 grads["b2"] = np.sum(grad_Z2, axis = 0)
 grads["W2"] = O1.T.dot(grad_Z2)
 grads["W2"] += 2 * self.reg * W2

 grad_O1 = grad_Z2.dot(W2.T)
 grad_O1[O1 <= 0] = 0 #the RELU function.

 grads["b1"] = np.sum(grad_O1, axis = 0)
 grads["W1"] = X.T.dot(grad_O1)
 grads["W1"] += 2 * self.reg * W1
 #####
 # END OF YOUR CODE #
 #####

 return loss, grads
```



# Pytorch Implementation

- Pytorch implementation vs Numpy implementation (ours)

```
def train_part34(model, optimizer, epochs=1):
 """
 Train a model on CIFAR-10 using the PyTorch Module API.

 Inputs:
 - model: A PyTorch Module giving the model to train.
 - optimizer: An Optimizer object we will use to train the model
 - epochs: (Optional) A Python integer giving the number of epochs to train for

 Returns: Nothing, but prints model accuracies during training.
 """
 model = model.to(device=device) # move the model parameters to CPU/GPU
 for e in range(epochs):
 for t, (x, y) in enumerate(loader_train):
 model.train() # put model to training mode
 x = x.to(device=device, dtype=dtype) # move to device, e.g. GPU
 y = y.to(device=device, dtype=torch.long)

 scores = model(x)
 loss = F.cross_entropy(scores, y)

 # Zero out all of the gradients for the variables which the optimizer
 # will update.
 optimizer.zero_grad()

 # This is the backwards pass: compute the gradient of the loss with
 # respect to each parameter of the model.
 loss.backward()

 # Actually update the parameters of the model using the gradients
 # computed by the backwards pass.
 optimizer.step()

 if t % print_every == 0:
 print('Iteration %d, loss = %.4f' % (t, loss.item()))
 check_accuracy_part34(loader_val, model)
 print()
```

```
hidden_layer_size = 4000
learning_rate = 1e-2
model = TwoLayerFC(3 * 32 * 32, hidden_layer_size, 10)
optimizer = optim.SGD(model.parameters(), lr=learning_rate)

train_part34(model, optimizer)
```

```
num_train = self.X_train.shape[0]
iterations_per_epoch = max(num_train // self.batch_size, 1)
num_iterations = self.num_epochs * iterations_per_epoch

for t in range(num_iterations):
 self._step()

 # Maybe print training loss
 if self.verbose and t % self.print_every == 0:
 print('(Iteration %d / %d) loss: %f' % (
 t + 1, num_iterations, self.loss_history[-1]))

 # At the end of every epoch, increment the epoch counter and decay
 # the learning rate.
 epoch_end = (t + 1) % iterations_per_epoch == 0
 if epoch_end:
 self.epoch += 1
 for k in self.optim_configs:
 self.optim_configs[k]['learning_rate'] *= self.lr_decay

 # Check train and val accuracy on the first iteration, the last
 # iteration, and at the end of each epoch.
 first_it = (t == 0)
 last_it = (t == num_iterations - 1)
 if first_it or last_it or epoch_end:
 train_acc = self.check_accuracy(self.X_train, self.y_train,
 num_samples=self.num_train_samples)
 self.train_acc_history.append(train_acc)
 if self.X_val is not None:
 val_acc = self.check_accuracy(self.X_val, self.y_val,
 num_samples=self.num_val_samples)
 self.val_acc_history.append(val_acc)

 if self.verbose:
 print('(Epoch %d / %d) train acc: %f; val_acc: %f' % (
 self.epoch, self.num_epochs, train_acc, val_acc))

 # Keep track of the best model
 if self.X_val is not None and val_acc > self.best_val_acc:
 self.best_val_acc = val_acc
 self.best_params = {}
 for k, v in self.model.params.items():
 self.best_params[k] = v.copy()

At the end of training swap the best params into the model
```

**Thank You**